Universidad de San Carlos de Guatemala
Centro Universitario de Occidente División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2
Mario Ramirez
Sección: A



Proyecto Fase 2

Carlos Raúl Alberto López Peláez 202031871

Quetzaltenango, 27 de Octubre del 2026

Índice

1. Información General..... 4
1.1. Descripción del Proyecto.....4
2. Requisitos del Entorno y Dependencias..... 5
2.1. Entorno de Ejecución y Herramientas.....5
2.2. Generadores de Analizadores y Librerías.....5
3. Compilación, Generación y Ejecución.....7
3.1. Regeneración de Scanners y Parsers (ANTLR4)..... 7
3.2. Compilación del Proyecto con Maven.....7
3.3. Ejecución de la Interfaz Gráfica..... 7
4. Estructura del Proyecto.....8
4.1. Módulo de Orquestación y Multilenguaje (org.compi2.orquestador).....8
4.3. Módulo del Árbol de Sintaxis Abstracta (org.compi2.ast).....10
4.5. Módulo de Generación de Código Intermedio (org.compi2.c3d).....12
4.6. Módulo de Interfaz Gráfica y Visualización (org.compi2.UI y org.compi2).....13
4.7. Módulo de Gestión de Errores (org.compi2.errores).....14
4.8. Módulo de Generación de Código Ensamblador RISC-V..... 15
5. Análisis Semántico y Estructuras Centrales.....15
5.1. Árbol de Sintaxis Abstracta (AST).....15
Jerarquía de Expresiones (org.compi2.ast.expresiones)..... 15
Jerarquía de Sentencias (org.compi2.ast.sentencias)..... 16
6. Diagrama de Clases del Sistema..... 17
Links:.....18
Diagrama de clases resumido.....18
Diagrama de Clases.....18
7. Diagrama de Arquitectura..... 19
Links:.....20
Arquitectura resumida..... 20
Arquitectura Completa.....20
9. Especificaciones de Palabras Reservadas, Símbolos y Gramática.....21
1. Especificaciones de Palabras Reservadas, Símbolos y Gramática.....21
1.1 Lenguaje Pig Latin (.pig)..... 21
Palabras Reservadas..... 21
Símbolos y Operadores.....21
Estructura Gramatical Base.....22
1.2. Lenguaje Y (.y)..... 23
Palabras Reservadas..... 23
Símbolos y Operadores.....23
Estructura Gramatical Base (Estilo Python con Sangría).....24
1.3. Lenguaje Zetariano (.z)..... 25
Palabras Reservadas..... 25

Símbolos y Operadores....25
Estructura Gramatical Base (Estilo Java / C#)....26
2. Equivalencia de Tipos entre Lenguajes....27
3. Tablas de Compatibilidad de Tipos....27
3.1. Operador Suma (+) y Concatenación de Cadenas....27
3.2. Operaciones Aritméticas: Resta (-), Multiplicación (*), División (/), Módulo (%). 28
3.3. Operaciones Relacionales: Menor (<), Mayor (>), Menor o Igual (<=), Mayor o Igual (>=)....28
3.4. Operaciones de Igualdad (==) y Diferencia (!=)....29
3.5. Operaciones Lógicas: Conjunción (&&), Disyunción (||), Negación (!)....30
3.6. Reglas de Asignación y Compatibilidad (Destino = Valor)....30
10. Programación Orientada a Objetos y Generación de Código RISC-V.... 31
10.1. Reglas Semánticas de Herencia, Visibilidad y Sobrescritura....31
10.2. Estructura de Objetos en el Heap y Despacho Dinámico....31
10.3. Traducción a Ensamblador RISC-V y Punto Fijo....32

1. Información General

1.1. Descripción del Proyecto

El presente software implementa un compilador multilenguaje. El sistema es capaz de procesar de manera concurrente tres lenguajes con paradigmas sintácticos y semánticos distintos:

1. **PigLatin (.pig)**: Lenguaje principal estructurado y operadores específicos de flujo de entrada/salida.

2. **Y (.y)**: Lenguaje modular estructurado basado en sangrías e indentación semántica obligatoria, orientado a la definición de estructuras de datos y funciones.

3. **Zetariano (.z**): Lenguaje orientado a objetos que implementa clases, constructores parametrizados y por defecto, métodos de instancia con tipado de retorno o void, campos de estado con inicializadores y encapsulamiento.

Existe un Orquestador de Compilación que resuelve las dependencias de importación entre archivos import, permitiendo llamadas cruzadas donde un programa en PigLatin puede instanciar clases de Zetariano, invocar sus constructores y métodos, o consumir funciones y estructuras exportadas por módulos del lenguaje Y. El resultado final del proceso es la traducción a Código de Tres Direcciones.

En la Fase 2, el lenguaje orientado a objetos Zetariano incorpora soporte para herencia simple mediante extends, modificadores de visibilidad public, protected, default y private, sobrescritura obligatoria de métodos con la anotación @Override, retornos covariantes, sobrecarga de constructores y métodos, e invocación a miembros de la superclase mediante super. Asimismo, el flujo de compilación añade la etapa de generación de código ensamblador RISC-V, traduciendo las cuartetas de Código de Tres Direcciones a instrucciones nativas de 32 bits con despacho dinámico polimórfico y aritmética decimal en punto fijo.

2. Requisitos del Entorno y Dependencias

2.1. Entorno de Ejecución y Herramientas

Componente	Requisito Mínimo	Versión Utilizada en el Proyecto
Java Development Kit (JDK)	JDK 21 LTS	Microsoft Build of
Automatización de Construcción	Apache Maven 3.8+	Apache Maven 3.9.12
Entorno de Desarrollo (IDE)	IntelliJ IDEA 2024+	IntelliJ IDEA 2026.2.2
Sistema Operativo	Windows 10/11 x64 / Linux x64	Microsoft Windows 11 Home Single Language (64 bits)
Compilador C (para C3D)	GCC 9.0+ / Clang 10+	GCC 13.x / 14.x (MinGW-w64)

2.2. Generadores de Analizadores y Librerías

El archivo pom.xml gestiona las siguientes dependencias oficiales:

```xml
<dependencies>
      <!-- Motor del Parser y Generador Léxico -->
      <dependency>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4</artifactId>
      <version>4.13.2</version>
      <scope>compile</scope>
      </dependency>

      <!-- Generación de código boilerplate (Getters,
Constructores) -->
      <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.36</version>
      <scope>provided</scope>
      </dependency>

      <!-- Framework de Interfaz Gráfica Moderna -->
      <dependency>
```

```
      <groupId>com.formdev</groupId>
      <artifactId>flatlaf</artifactId>
      <version>3.7.2</version>
      </dependency>
      <dependency>
      <groupId>com.formdev</groupId>
      <artifactId>flatlaf-intellij-themes</artifactId>
      <version>3.7.2</version>
      </dependency>

      <!-- Motor Embebido de Graphviz (Renderizado AST sin
dot.exe externo) -->
      <dependency>
      <groupId>guru.nidi</groupId>
      <artifactId>graphviz-java</artifactId>
      <version>0.18.1</version>
      </dependency>

      <!-- Proveedor de Logging Silencioso -->
      <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>1.7.36</version>
      <scope>runtime</scope>
      </dependency>

      <!-- Framework de Pruebas Automatizadas -->
      <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
      </dependency>
 </dependencies>
```

3. Compilación, Generación y Ejecución

3.1. Regeneración de Scanners y Parsers (ANTLR4)

Las gramáticas están ubicadas en src/main/antlr4/gramaticas/. Para regenerar las clases Java:

```shell
mvn antlr4:antlr4
```

Este comando procesa las gramáticas piglatin.g4, y.g4 y zetariano.g4, ubicando los analizadores resultantes en target/generated-sources/antlr4/.

3.2. Compilación del Proyecto con Maven

Para compilar el código fuente y procesar las anotaciones de Lombok:

```shell
mvn clean compile
```

Para empaquetar la aplicación en un archivo .jar:

```shell
mvn clean package
```

3.3. Ejecución de la Interfaz Gráfica

Se puede iniciar la aplicación directamente mediante el plugin de ejecución de Maven o mediante la clase principal:

```shell
# Mediante Maven
mvn exec:java

# O ejecutando la clase compilada directamente con Java
java -cp "target/classes;target/dependency/*" org.compi2.Main
```

4. Estructura del Proyecto

4.1. Módulo de Orquestación y Multilenguaje (org.compi2.orquestador)

- **OrquestadorCompilacion.java:**
    - **Rol:** Coordinador de toda la compilación.
    - **Responsabilidad:** Gestiona el ciclo de la compilación a partir de un archivo raíz .pig. Resuelve de forma recursiva los grafos de dependencias a través de GestorImports, delega el análisis sintáctico a los parsers respectivos, invoca los constructores del AST, coordina las dos pasadas del análisis semántico y en caso de no existir errores fatales, dispara la generación de C3D.
    - **Métodos principales:**
        - **compilarArchivo(Path rutaArchivoPig)**: Punto de entrada que orquesta la lectura, el análisis sintáctico , el análisis semántico y la generación de C3D.
        - **compilarPigLatin()**: Instancia el analizador léxico el analizador sintáctico y el receptor de errores para PigLatin.
        - **compilarModuloY(**): Aplica el preprocesador Y SangriaToken antes de analizar los módulos .y.
        - **compilarModuloZeta()**: Analiza sintácticamente los archivos .zeta e instancia su correspondiente constructor del AST.
        - **ejecutarAnalisisSemanticoPasada1()** y **ejecutarAnalisisSemanticoPasada2()**: Separan el registro de la validación estricta de cuerpos de función e instrucciones.
- **GestorImports.java:**
    - **Rol:** Descubrimiento, validación y prevención de ciclos en dependencias.
    - **Responsabilidad:** Lee los import presentes en el código, normaliza rutas relativas y absolutas, detecta la extensión, comprueba que los archivos existan en el sistema de archivos.

- **ArchivoFuente.java:**
    - **Rol:** Modelo para cada traducción.
    - **Responsabilidad:** Encapsula el nombre del archivo, su ruta en disco, su contenido, el lenguaje identificado.

- **ResultadoCompilacion.java:**
    - **Rol:** Objeto resultado de compilación.
    - **Responsabilidad:** Devuelve a la interfaz gráfica el estado final de la compilación: indicador booleano de éxito, cadena con el código C3D, mapa de todos los archivos importados, la instancia completa de la TablaSimbolos y la lista de errores.

4.2. Módulo de Gramáticas y Constructores de AST (org.compi2.constructores_ast y src/main/antlr4)

- **piglatin.g4, y.g4, zetariano.g4:**
    - **Rol:** Especificaciones formales léxicas y sintácticas en ANTLR4.
    - **Responsabilidad:** Definen la tokenización mediante expresiones regulares y las reglas gramaticales.

- **YSangriaToken.java (org.compi2.analisis_semantico):**
    - **Rol:** Procesador léxico de indentación.
    - **Responsabilidad:** Emula el comportamiento del lexer de Python para el lenguaje Y.

- **ConstructorAstPigLatin.java:**
    - **Rol:** Visitor traductor de PigLatin hacia el AST común.
    - **Responsabilidad:** Sobrescribe los métodos de visita generados por ANTLR para instanciar los nodos.

- **ConstructorAstY.java:**
    - **Rol:** Visitor traductor del lenguaje Y.
    - **Responsabilidad:** Procesa las secciones estructuras: y funciones: de Y, convirtiendo las definiciones en nodos.

- **ConstructorAstZetariano.java:**
    - **Rol:** Visitor traductor del lenguaje orientado a objetos Zetariano.
    - **Responsabilidad:** Procesa las clases, sus constructores y sus métodos de instancia.

4.3. Módulo del Árbol de Sintaxis Abstracta (org.compi2.ast)

- **NodoAST.java:**
    - **Rol:** Interfaz base de todos los elementos del AST.
    - **Responsabilidad:** Define el obtener la línea y columna de cada nodo en el código fuente original para trazabilidad y reporte de errores.

- **Subpaquete org.compi2.ast.expresiones:**

    - **NodoOperacionBinaria.java:** Expresiones con operador izquierdo y derecho .
    - **NodoOperacionUnaria.java:** Negaciones lógicas, cambios de signo y operadores de post-incremento/decremento.
    - **NodoLiteral.java:** Almacena valores literales primitivos.
    - **NodoIdentificador.java:** Referencias por nombre a variables, constantes o parámetros.
    - **NodoLlamadaFuncion.java y NodoLlamadaMetodo.java:** Funciones globales o métodos.
    - **NodoAccesoMiembro.java y NodoAccesoArreglo.java:** Lectura de campos de clases/estructuras y lectura de arreglos.
    - **NodoInstanciacionObjeto.java y NodoInstanciacionArreglo.java:** Construcción con el operador nuevo o novus.
    - **NodoExpresionTernaria.java:** Expresión condicional ternaria con unificación de tipos numéricos y jerarquías de clases.
    - **NodoLlamadaMetodoSuper.java:** Invocación explícita a métodos heredados de la superclase.

- **Subpaquete org.compi2.ast.sentencias:**

    - **NodoDeclaracion.java y NodoAsignacion.java:** Creación de variables y asignación de expresiones en Stack.
    - **NodoIf.java, NodoWhile.java, NodoDoWhile.java, NodoFor.java, NodoSwitch.java:** Estructuras de condicional, ciclos y selección múltiple.
    - **NodoClase.java, NodoEstructura.java, NodoFuncion.java:** Nodos de declaración de tipos de usuario y bloques de funciones ejecutables.
    - **NodoImprimir.java y NodoLectura.java:** Sentencias de entrada/salida correspondientes a los operadores >> y << de PigLatin.
    - **NodoLlamadaSuper.java:** Invocación al constructor de la superclase como primera instrucción del constructor hijo.
    - **NodoCaso.java:** Representación de bloques case y default dentro de estructuras de selección múltiple.

4.4. Módulo de Análisis Semántico y Tipos (org.compi2.analisis_semantico y org.compi2.tipos)

- **TablaSimbolos.java:**
    - **Rol:** Estructura de datos.
    - **Responsabilidad:** Entorno jerárquico mediante Ámbitos. Permite abrir y cerrar ámbitos, insertar símbolos, gestionar funciones y métodos, y exportar un listado plano con offsets.

- **Ambito.java:**
    - **Rol:** Representación de un nivel de entorno.
    - **Responsabilidad:** Mantiene el nombre del ámbito, la referencia al ámbito padre y un diccionario local.

- **Simbolo.java:**
    - **Rol:** Registro identificador en el compilador.
    - **Responsabilidad:** Almacena el nombre, tipo de dato, categoría, archivo fuente de origen y el desplazamiento de memoria,asignado para C3D.

- **ContadorDesplazamiento.java:**
    - **Rol:** Asignador de direcciones.
    - **Responsabilidad:** Administra contadores de offsets para:
        1. Variables globales.
        2. Parámetros y variables locales.
        3. Atributos y campos dentro de estructuras y objetos.

- **ComprobadorTipos.java:**
    - **Rol:** Evaluador de reglas de tipado.
    - **Responsabilidad:** Implementa los tipos para todas las operaciones binarias y unarias, verifica que el valor asignado a una variable sea compatible con su tipo declarado, valida que las expresiones condicionales sean booleanas y resuelve conversiones numéricas.

- **Tipo.java y TipoBase.java:**
    - **Rol:** Modelo integral del sistema de tipos de datos.
    - **Responsabilidad:** TipoBase enumera los primitivos y compuestos.

- **TipoClase.java, AtributoClase.java, MetodoClase.java y ConstructorClase.java:**
    - **Rol:** Metamodelo de clases, herencia y polimorfismo.

- **Responsabilidad:** TipoClase mantiene la referencia a su superclase, valida relaciones de subtipado, detecta ciclos de herencia y almacena atributos, constructores sobrecargados y métodos propios o sobrescritos.

- **Visibilidad.java:**

  - **Rol:** Enumeración de niveles de encapsulamiento.
  - **Responsabilidad:** Define los niveles PRIVATE, DEFAULT, PROTECTED y PUBLIC, e impide que un método sobrescrito con @Override reduzca la visibilidad declarada en la superclase.

4.5. Módulo de Generación de Código Intermedio (org.compi2.c3d)

- **GeneradorC3D.java:**
  - **Rol:** Generación de instrucciones C3D.
  - **Responsabilidad:** Recorre el AST y traduce cada instrucción de alto nivel a instrucciones de tres direcciones.

- **ContenedorC3D.java:**
  - **Rol:** Código intermedio.
  - **Responsabilidad:** Contiene la lista de Cuarteta.

- **Cuarteta.java:**
  - **Rol:** Representación de una instrucción de cuadrupla.
  - **Responsabilidad:** Encapsula el operador , argumento 1, argumento 2, resultado y comentario opcional. Genera la línea sintácticamente correcta en lenguaje C correspondiente a la operación.
- **GestorTemporales.java y GestorEtiquetas.java:**

  - **Rol:** Administradores de identificadores.
  - **Responsabilidad:** GestorTemporales genera variables temporales y emite su declaración como variables double globales de C. GestorEtiquetas genera etiquetas.

- **PlantillaC.java:**

  - **Rol:** Salida compatible con c.
  - **Responsabilidad:** Envuelve las cuartetas generadas dentro de un archivo C completo con punteros P y H, la función para impresión de cadenas dinámicas del Heap.

- **ContextoC3D.java:**

    - **Rol:** Administrador del diseño de memoria en STACK y HEAP.
    - **Responsabilidad:** Calcula la estructura física de cada clase reservando la primera posición del objeto en el HEAP para el identificador dinámico de tipo, seguido de los atributos heredados de la superclase y los atributos propios de la subclase.

- **GeneradorDeclaraciones.java, GeneradorSentencias.java y GeneradorExpresiones.java:**

    - **Rol:** Generadores modulares de declaraciones, tablas virtuales, sentencias y expresiones.
    - **Responsabilidad:** Emiten constructores encadenados con super, despachadores polimórficos para métodos sobrescritos, inicialización por defecto en referencias de objetos y escalado automático en punto fijo entre enteros y decimales.

4.6. Módulo de Interfaz Gráfica y Visualización (org.compi2.UI y org.compi2)

- **Main.java:**
    - **Rol:** Punto de entrada de la aplicación.
    - **Responsabilidad:** Configura el tema visual, redondeo de botones y componentes Swing, y despliega la VentanaPrincipal.

- **VentanaPrincipal.java:**

    - **Rol:** Controlador general de la interfaz gráfica del IDE.
    - **Responsabilidad:** Dispone en pantalla el explorador de archivos en árbol, el editor de código con pestañas, la consola de salida y el visor interactivo de resultados. Vincula los eventos de menú.

- **TareaCompilacion.java (org.compi2.UI.compilacion):**

    - **Rol:** Ejecución asíncrona en segundo plano de la compilacion.
    - **Responsabilidad:** Ejecuta la compilación en segundo plano para evitar que la UI se congele, enviando actualizaciones de estado a la consola y publicando los resultados finales en las tablas y visores gráficos.

- **RenderizadorGraphviz.java (org.compi2.UI.graficos):**

    - **Rol:** Renderizado de árboles AST a imágenes.
    - **Responsabilidad:** Utiliza la librería integrada para procesar cadenas DOT y generar directamente en memoria un BufferedImage.

- **GeneradorDot.java (org.compi2.UI.graficos):**
    - **Rol:** Constructor DOT de Graphviz.
    - **Responsabilidad:** Recorre el AST concreto y emite el código en lenguaje DOT con colores, formas y jerarquías claras para nodos terminales y no terminales.

- **PanelImagenZoom.java (org.compi2.UI.graficos):**
    - **Rol:** Componente Swing interactivo para imágenes.
    - **Responsabilidad:** Permite visualizar el gráfico del AST renderizado con soporte para zoom mediante la rueda del ratón y paneo mediante arrastre del mouse.

- **DialogoTabla.java, ModeloTablaSimbolos.java y ModeloTablaErrores.java (org.compi2.UI.tablas):**
    - **Rol:** Ventanas modales de datos.
    - **Responsabilidad:** Despliegan tablas ordenables con estilos FlatLaf para examinar la totalidad de los símbolos recolectados y los errores ocurridos durante el análisis.

4.7. Módulo de Gestión de Errores (org.compi2.errores)

- **CustomErrorListener.java:**
    - **Rol:** Interceptor de errores de ANTLR4.
    - **Responsabilidad:** Implementa ANTLRErrorListener para atrapar errores léxicos y sintácticos, extrayendo el archivo, línea, columna y mensaje explicativo para registrar instancias de ErrorCompilacion.

- **ErrorCompilacion.java y TipoError.java:**
    - **Rol:** Modelo de errores del compilador.
    - **Responsabilidad:** TipoError clasifica el error en LEXICO, SINTACTICO o SEMANTICO. ErrorCompilacion encapsula la descripción, ubicación espacial en el archivo y tipo de fallo para su presentación en consola y en tabla modal.

4.8. Módulo de Generación de Código Ensamblador RISC-V

- **GeneradorRiscV.java:**
    - **Rol:** Traductor de Código de Tres Direcciones a ensamblador RISC-V.
    - **Responsabilidad:** Recorre la lista de cuartetas generada en la fase intermedia y emite un programa ensamblador completo con segmentos de datos para literales de texto y arreglos globales STACK y HEAP, junto con el segmento de código para la rutina principal, constructores, métodos, despachadores virtuales y subrutinas nativas.

5. Análisis Semántico y Estructuras Centrales

5.1. Árbol de Sintaxis Abstracta (AST)

La jerarquía de clases del AST se fundamenta en la interfaz base:

```
public interface NodoAST {
        int getLinea();
        int getColumna();
}
```

Jerarquía de Expresiones (org.compi2.ast.expresiones)

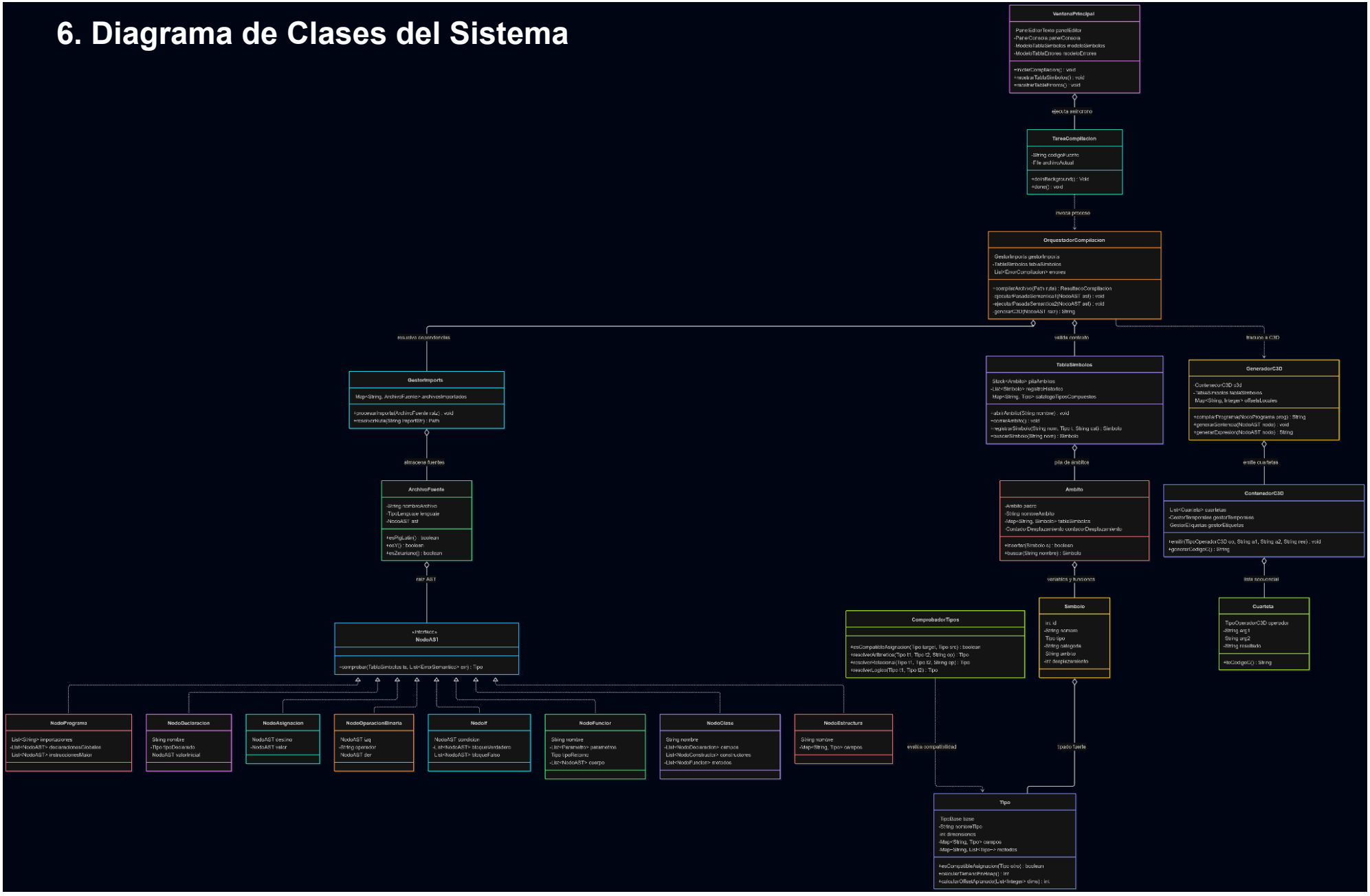
- **NodoOperacionBinaria:** Operaciones aritméticas, lógicas y relacionales.
- **NodoOperacionUnaria:** Negación lógica, menos unario, incremento y decremento.
- **NodoLiteral:** Valores constantes de tipo entero, decimal, cadena, carácter o booleano.
- **NodoIdentificador:** Referencias a variables o constantes en memoria.
- **NodoLlamadaFuncion y NodoLlamadaMetodo:** Invocación de subrutinas con pase de parámetro.
- **NodoAccesoMiembro:** Navegación por campos de objetos o estructuras.
- **NodoAccesoArreglo:** Lectura en memoria.
- **NodoInstanciacionObjeto y NodoInstanciacionArreglo:** Reserva de espacio en el Heap.
- **NodoExpresionTernaria:** Evaluación condicional con promoción numérica y unificación por superclase común.
- **NodoLlamadaMetodoSuper:** Invocación directa a métodos de la superclase.

Jerarquía de Sentencias (org.compi2.ast.sentencias)

- **NodoDeclaracion y NodoAsignacion:** Creación y modificación de celdas.
- **NodoIf, NodoWhile, NodoDoWhile, NodoFor:** Estructuras de bifurcación y ciclos.

- **NodoClase, NodoEstructura, NodoFuncion:** Definición de tipos de datos de usuario y subrutinas
- **NodoLlamadaSuper:** Invocación al constructor padre en la primera línea del constructor hijo.

6. Diagrama de Clases del Sistema

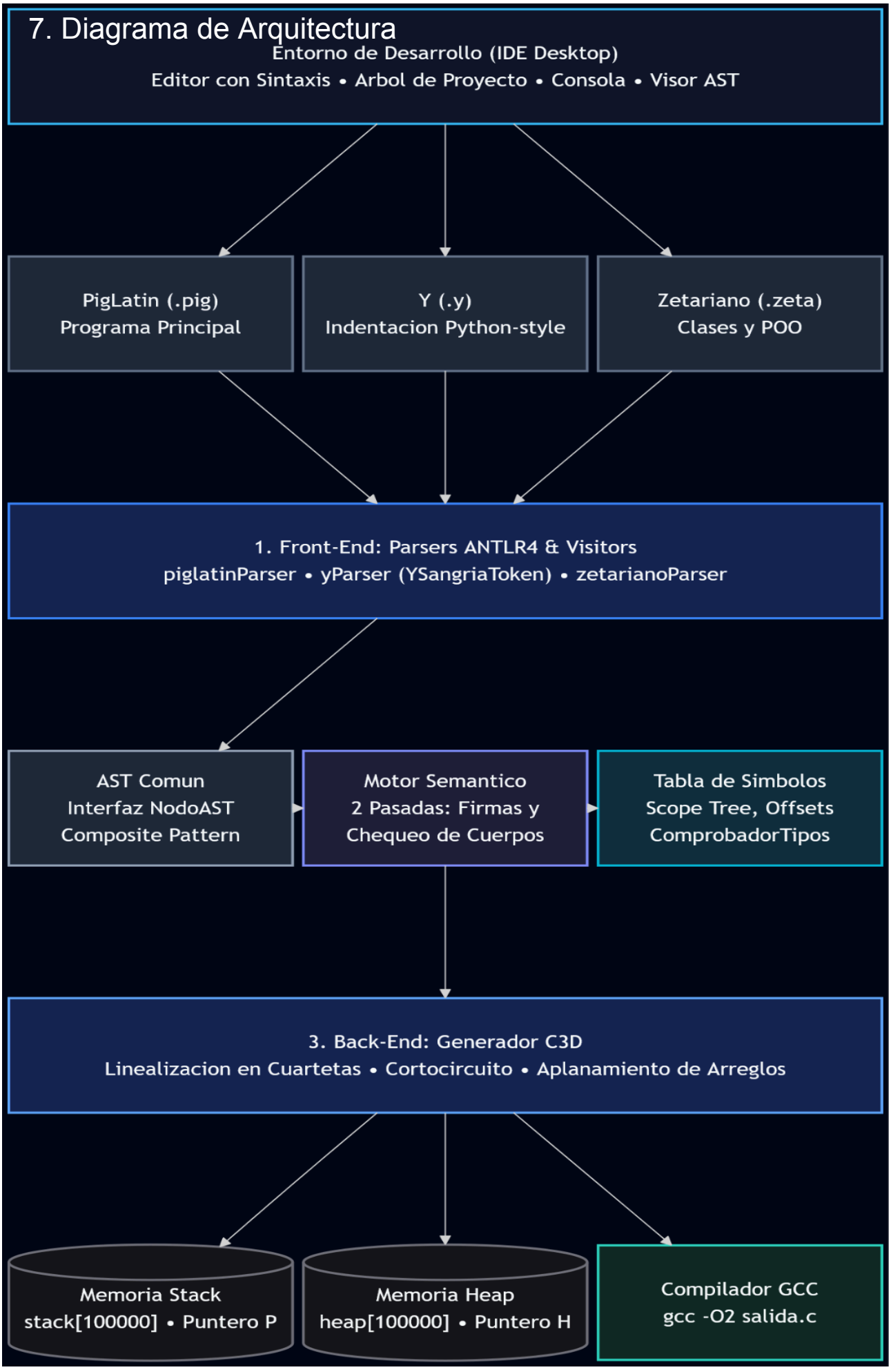


Links:

Diagrama de clases resumido

Diagrama de Clases

7. Diagrama de Arquitectura



Links:

Arquitectura resumida

Arquitectura Completa

9. Especificaciones de Palabras Reservadas, Símbolos y Gramática

1. Especificaciones de Palabras Reservadas, Símbolos y Gramática

1.1 Lenguaje Pig Latin (`.pig`)

Palabras Reservadas

Categoría	Palabras Clave
Estructura del Programa	import, VARIABILES > / VARIABLES >, MAIOR >, FINIS, finis
Declaraciones	esto, series, novus
Tipos de Datos	numerus (entero), decimalis (flotante), textum (cadena), littera (carácter), booleanus (booleano)
Literales Booleanos	verum, falsus
Control de Flujo	si, aliter, dum, facere, per, interrumpe / interrumpir, perge

Símbolos y Operadores

Tipo	Símbolos
Entrada / Salida	>> (imprimir), << (leer)
Aritméticos	+, -, *, /, ++, --
Relacionales	<, >, <=, >=
Igualdad	==, !=
Lógicos	&&, `
Asignación	=
Delimitadores	(,), {, }, [,], ;, ,, ., :
Comentarios	// (línea), ## ... ## (bloque)

Estructura Gramatical Base

```
programa: importacion* seccionVariables? seccionPrincipal ;
seccionVariables: VARIABLES declaracionGlobal* ;
seccionPrincipal: MAIOR instruccion* FIN_PROGRAMA PUNTO_Y_COMA? ;
declaracionGlobal
    : ESTO ID DOS_PUNTOS tipo (ASIGNACION expresion)?
PUNTO_Y_COMA
    | ESTO ID DOS_PUNTOS NOVUS ID PAR_IZQ listaArgumentos?
PAR_DER DOS_PUNTOS ID PUNTO_Y_COMA
    | SERIES ID CORCHETE_IZQ expresion CORCHETE_DER DOS_PUNTOS
tipo (ASIGNACION listaValores)? PUNTO_Y_COMA
    ;
instruccion
    : asignacion PUNTO_Y_COMA
    | llamadaMetodo PUNTO_Y_COMA
    | impresion PUNTO_Y_COMA
    | condicionalSi
    | cicloDum
    | cicloFacere
    | cicloPer
    | (INTERRUMPIR | PERGE) PUNTO_Y_COMA
    ;
```

1.2. Lenguaje Y (.y)

Palabras Reservadas

Categoría	Palabras Clave
Estructura Modular	%estructuras, %funciones, estructura, definir
Tipos de Datos	entero, flotante, caracter, bool, cadena
Literales Booleanos	verdadero, falso
Control de Flujo	si, entonces, sino, contrario, elegir, caso, siempre, mientras, hacer, para, romper, continuar, retornar
Funciones Especiales	imprimir, leer

Símbolos y Operadores

Tipo	Símbolos
Aritméticos	+, -, *, /, ++, --
Relacionales	<, >, <=, >=
Igualdad	==, !=
Lógicos	&&, `
Asignación y Retorno	=, ->
Delimitadores	(,), {, }, [,], ;, ,, ., :
Tokens Virtuales	SANGRIA, FIN_SANGRIA
Comentarios	//, /* ... */

Estructura Gramatical Base (Estilo Python con Sangría)

JSON

```
programa: seccionEstructuras? seccionFunciones? EOF ;
seccionEstructuras: SEC_ESTRUCTURAS definicionEstructura* ;
definicionEstructura: ESTRUCTURA ID DOS_PUNTOS NUEVA_LINEA+
SANGRIA campoEstructura+ FIN_SANGRIA ;
seccionFunciones: SEC_FUNCIONES definicionFuncion* ;
definicionFuncion: DEFINIR? ID PAR_IZQ parametros? PAR_DER
(FLECHA tipo)? DOS_PUNTOS bloque ;
bloque: NUEVA_LINEA+ SANGRIA (instruccion | NUEVA_LINEA)*
FIN_SANGRIA ;
condicionalSi: SI PAR_IZQ expresion PAR_DER ENTONCES bloque (SINO
PAR_IZQ expresion PAR_DER ENTONCES bloque)* (CONTRARIO bloque)? ;
seleccionElegir: ELEGIR PAR_IZQ expresion PAR_DER DOS_PUNTOS
bloqueElegir ;
bloqueElegir: NUEVA_LINEA+ SANGRIA (seccionCaso | seccionSiempre
| NUEVA_LINEA)* FIN_SANGRIA ;
seccionCaso: CASO literal DOS_PUNTOS bloque ;
seccionSiempre: SIEMPRE DOS_PUNTOS bloque ;
```

1.3. Lenguaje Zetariano (.z)

Palabras Reservadas

Categoría	Palabras Clave
Estructura POO	public, class, void, new, null
Tipos de Datos	int, double, char, boolean, String
Literales Booleanos	true, false
Control de Flujo	if, else, switch, case, default, for, while, do, break, continue, return
Entrada / Salida	print, println, readln

Símbolos y Operadores

Tipo	Símbolos
Aritméticos y Asignación Compuesta	+, -, *, /, %, ++, --, +=, -=, *=, /=, %=
Relacionales	<, >, <=, >=
Igualdad	==, !=
Lógicos y Operador Ternario	&&, `
Asignación	=
Delimitadores	(,), {, }, [,], ;, ,, ., :
Comentarios	//, /* ... */

Estructura Gramatical Base (Estilo Java / C#)

JSON

```
programa: declaracionClase EOF ;
declaracionClase: PUBLICO? CLASE ID LLAVE_IZQ miembroClase*
LLAVE_DER ;
miembroClase
    : declaracionAtributo
    | declaracionConstructor
    | declaracionMetodo
    ;
declaracionMetodo: PUBLICO? tipoRetorno ID PAR_IZQ parametros?
PAR_DER bloque ;
modificadorAcceso: PUBLICO | PROTEGIDO | PRIVADO ;
declaracionConstructor: modificadorAcceso? ID PAR_IZQ parametros?
PAR_DER LLAVE_IZQ llamadaSuper? instruccion* LLAVE_DER ;
llamadaSuper: SUPER PAR_IZQ listaArgumentos? PAR_DER PUNTO_Y_COMA
;
bloque: LLAVE_IZQ instruccion* LLAVE_DER ;
instruccion
    : declaracionLocal PUNTO_Y_COMA
    | asignacion PUNTO_Y_COMA
    | impresion PUNTO_Y_COMA
    | cicloIf
    | seleccionSwitch
    | cicloFor
    | cicloWhile
    | cicloDoWhile
    | (ROMPER | CONTINUAR | RETORNAR expresion?) PUNTO_Y_COMA
    ;
```

2. Equivalencia de Tipos entre Lenguajes

Tipo Semántico Universal	Pig Latin (.pig)	Y (.y)	Zetariano (.z)	Jerarquía Numérica
Entero	numerus	entero	int	3
Decimal / Flotante	decimalis	flotante	double	4
Carácter	littera	caracter	char	2 (Código ASCII)
Booleano	booleanus	bool	boolean	1
Cadena de Texto	textum	cadena	String	5
Referencia Nula / Vacío	null	(nulo implícito)	null / void	-1

3. Tablas de Compatibilidad de Tipos

3.1. Operador Suma (+) y Concatenación de Cadenas

Regla de Concatenación: Si cualquiera de los dos operandos es una Cadena (String / cadena / textum), la operación se convierte en concatenación y el resultado resultante siempre es Cadena.

Operando 1 \ 2	Entero (int)	Decimal (double)	Carácter (char)	Booleano (bool)	Cadena (String)
Entero (int)	Entero	Decimal	Entero	**ERROR**	Cadena
Decimal (double)	Decimal	Decimal	Decimal	**ERROR**	Cadena
Carácter (char)	Entero	Decimal	Entero	**ERROR**	Cadena
Booleano (bool)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	Cadena
Cadena (String)	Cadena	Cadena	Cadena	Cadena	Cadena

3.2. Operaciones Aritméticas: Resta (-), Multiplicación (*), División (/), Módulo (%)

> Regla Aritmética: No admiten cadenas ni booleanos bajo ninguna circunstancia. Entre tipos numéricos predomina el de mayor jerarquía.

Operando	**Entero (int)**	**Decimal (double)**	**Carácter (char)**	**Booleano (bool)**	**Cadena (String)**
Entero (int)	Entero	Decimal	Entero	**ERROR**	**ERROR**
Decimal (double)	Decimal	Decimal	Decimal	**ERROR**	**ERROR**
Carácter (char)	Entero	Decimal	Entero	**ERROR**	**ERROR**
Booleano (bool)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	**ERROR**
Cadena (String)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	**ERROR**

3.3. Operaciones Relacionales: Menor (<), Mayor (>), Menor o Igual (<=), Mayor o Igual (>=)

> Regla Relacional: Solo son válidas entre tipos puramente numéricos (Entero, Decimal, Carácter). El tipo devuelto siempre es Booleano.

Operando 1 \ 2	**Entero (int)**	**Decimal (double)**	**Carácter (char)**	**Booleano (bool)**	**Cadena (String)**
Entero (int)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**
Decimal (double)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**
Carácter (char)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**
Booleano (bool)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	**ERROR**
Cadena (String)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	**ERROR**

3.4. Operaciones de Igualdad (==) y Diferencia (!=)

Regla de Igualdad:

1. Son válidas entre dos operandos del mismo tipo exacto (Cadena == Cadena, Booleano == Booleano).
2. Son válidas entre cualquier par de tipos numéricos (Entero == Decimal, Entero == Carácter).
3. Son válidas para comparar cualquier referencia a Estructura, Objeto o Arreglo contra null.
4. Cualquier comparación entre tipos heterogéneos incompatibles (ej. Entero == Cadena o Booleano == Entero) produce un ERROR.

Operando	**Entero (int)**	**Decimal (double)**	**Carácter (char)**	**Booleano (bool)**	**Cadena (String)**	**Referencia / Null**
Entero (int)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**	**ERROR**
Decimal (double)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**	**ERROR**
Carácter (char)	Booleano	Booleano	Booleano	**ERROR**	**ERROR**	**ERROR**
Booleano (bool)	**ERROR**	**ERROR**	**ERROR**	Booleano	**ERROR**	**ERROR**
Cadena (String)	**ERROR**	**ERROR**	**ERROR**	**ERROR**	Booleano	**ERROR**
Referencia / Null	**ERROR**	**ERROR**	**ERROR**	**ERROR**	**ERROR**	Booleano

3.5. Operaciones Lógicas: Conjunción (&&), Disyunción (||), Negación (!)

Expresión	Tipo de Operando(s)	Tipo de Retorno	Descripción
a && b	Booleano × Booleano	Booleano	Verdadero únicamente si ambos son verdaderos
a || b	Booleano × Booleano	Booleano	Verdadero si al menos uno es verdadero
!a	Booleano	Booleano	Invierte el valor lógico
Cualquier otro tipo	No booleano	**ERROR**	Error semántico de incompatibilidad de tipos

3.6. Reglas de Asignación y Compatibilidad (Destino = Valor)

En una asignación o paso de parámetros, se permite la promoción implícita sin pérdida de precisión. Si la asignación produce un estrechamiento (posible truncamiento) o los tipos son incompatibles, se genera un Error Semántico.

Tipo Destino (Variable)	Tipos Asignables Permitidos	Tipos Rechazados (ERROR)
Decimal (double)	Decimal, Entero, Carácter	Booleano, Cadena, Objetos, Arreglos
Entero (int)	Entero, Carácter (ASCII)	Decimal, Booleano, Cadena, Objetos
Carácter (char)	Carácter	Entero, Decimal, Booleano, Cadena
Booleano (bool)	Booleano	Entero, Decimal, Carácter, Cadena
Cadena (String)	Cadena	Tipos primitivos directos (requieren concatenación previa)
Objeto / Estructura	Mismo nombre de clase/estructura o null	Cualquier tipo primitivo u objeto de clase distinta
Arreglo (T[])	Arreglo del mismo tipo base y dimensiones, o null	Arreglos de distinto tipo/dimensión

10. Programación Orientada a Objetos y Generación de Código RISC-V

10.1. Reglas Semánticas de Herencia, Visibilidad y Sobrescritura

El analizador semántico valida las jerarquías de clases en dos pasadas bajo las siguientes reglas:

- **Herencia Simple y Prevención de Ciclos:** Cada clase en Zetariano puede heredar de una única superclase mediante la palabra reservada extends, verificando en la primera pasada que no existan ciclos directos ni indirectos en la jerarquía.
- **Visibilidad Pública y Protegida:** Los miembros declarados como public son accesibles desde cualquier clase y desde el archivo principal, mientras que los miembros protected solo son accesibles dentro de la misma clase y dentro de sus subclases directas o indirectas.
- **Visibilidad por Defecto y Privada:** Al no existir paquetes en el lenguaje, tanto los miembros sin modificador de acceso como los declarados con private son accesibles exclusivamente dentro de la propia clase que los declara.
- **Sobrescritura Obligatoria y Retornos Covariantes:** Todo método de una subclase que coincida en nombre y firma con un método público o protegido de su superclase debe incluir la anotación @Override, mantener o ampliar la visibilidad original y retornar el mismo tipo o una subclase del tipo declarado en la superclase.
- **Constructores e Invocación a la Superclase:** La instrucción super debe ubicarse como primera sentencia del constructor hijo; si se omite, el compilador verifica que la superclase disponga de un constructor sin parámetros accesible.

10.2. Estructura de Objetos en el Heap y Devolución Dinámica

Cada instancia creada mediante new o novus se almacena en un bloque contiguo del arreglo HEAP con la siguiente distribución:

- **Identificador Dinámico de Tipo:** La primera celda del objeto en el HEAP almacena el identificador entero único de la clase concreta instanciada en tiempo de ejecución.
- **Atributos Heredados y Propios:** Los atributos heredados conservan los mismos desplazamientos que en la superclase, seguidos de los campos propios añadidos por la subclase.
- **Despacho Virtual Polimórfico:** Al invocar un método sobre una referencia de tipo padre, el código intermedio llama a la tabla virtual correspondiente, la cual consulta el identificador dinámico del objeto en el HEAP y salta a la implementación sobrescrita de la subclase real.

10.3. Traducción a Ensamblador RISC-V y Punto Fijo

El generador traduce las cuartetas de tres direcciones a instrucciones RV32IM de 32 bits empleando la siguiente organización:

- **Punteros de Pila y Montículo:** El registro s0 actúa como puntero de marco sobre el arreglo STACK y el registro s1 actúa como puntero de asignación dinámica sobre el arreglo HEAP, mientras que s2 y s3 mantienen las direcciones base de ambos segmentos en memoria.
- **Registros Temporales y de Sistema:** Los registros t0 a t6 evalúan expresiones aritméticas, relacionales y desplazamientos de memoria, el registro ra almacena la dirección de retorno de subrutinas y los registros a0 y a7 gestionan las llamadas al sistema para entrada y salida.
- **Aritmética Decimal en Punto Fijo:** Los valores decimales se representan como enteros escalados por diez mil. En operaciones mixtas entre enteros y decimales, el compilador escala automáticamente el operando entero antes de operar, y la rutina nativa de impresión separa la parte entera y la fracción mediante división y módulo.